

Week 7 Assignment Report

Data Cleaning with Pandas & Incremental Processing with Delta Lake

Tools used: Python, Pandas, PySpark, Delta Lake, Databricks

1. Objective

The objective of this Week 7 assignment is to build a small but complete data engineering workflow that covers two stages of a real pipeline: cleaning raw data with Pandas, and then loading that cleaned data into Delta Lake to practice incremental (upsert) processing using the MERGE operation. The idea is to simulate what actually happens in a company when a new batch of data — containing both updated and brand-new records — needs to be merged into an existing table without duplicating or overwriting good data by mistake.

By the end of this assignment, the following skills were practiced:

- Reading, exploring and cleaning a CSV file using Pandas
- Handling missing values sensibly instead of just dropping everything
- Creating a derived column from existing columns
- Loading data into a Delta Table using PySpark on Databricks
- Performing an incremental load using the Delta Lake MERGE INTO command
- Validating the final dataset for row count and duplicate records

2. Introduction

Every data pipeline starts with raw data, and raw data is almost never clean. In this assignment, the starting point was a customer purchase dataset (`customer_master.csv`) containing customer details, the product purchased, price and quantity. This file intentionally had a few missing values and duplicate rows, similar to what happens in real systems due to manual entry errors, failed uploads, or repeated syncs.

The first part of the assignment used Pandas, which is the standard Python library for working with tabular data on a single machine. Pandas is good for small to medium datasets and is usually the first step before moving to a distributed tool, and it is also the easiest way to get a quick feel for a dataset before writing any bigger pipeline code.

The second part moved into Databricks and PySpark, since that is what large-scale data engineering pipelines actually use in production. Instead of just overwriting the whole table every time new data arrives, this assignment focused on incremental processing, meaning only the changed or new records are processed, which is far more efficient for big datasets. In a real company, tables can have millions of rows, so reloading everything every day is simply not practical — incremental loading using MERGE is the standard way around this.

This report walks through both parts step by step, in the same order they were done in the notebook, followed by a deeper explanation of the MERGE operation, validation checks, and what was learned along the way.

3. Tools and Technologies Used

The following tools and technologies were used to complete this assignment. Each one plays a different role in the overall workflow, from initial data cleaning to the final incremental load.

- Python – the base programming language used for the entire assignment
- Pandas – used in Part 1 for reading, exploring and cleaning the raw CSV file
- PySpark – the Python API for Apache Spark, used in Part 2 to work with larger-scale, distributed data
- Delta Lake – the storage layer used on top of Spark to get transaction support and the MERGE feature
- Databricks – the platform where the PySpark and Delta Lake code is actually meant to run, using Databricks Volumes to store the CSV files
- Jupyter Notebook – used to write and document the code in a step-by-step, explained format

Together, these tools represent a fairly typical (if simplified) data engineering stack: a lightweight tool for initial cleaning, and a distributed engine with a transactional storage layer for the actual pipeline work.

4. What is Delta Lake?

Delta Lake is an open-source storage layer that sits on top of regular Parquet files and adds a transaction log on top of them. This transaction log is what gives Delta Lake its most useful properties, especially for data engineering work:

- ACID transactions – so a write either fully succeeds or fully fails, there is no half-written table
- Schema enforcement – prevents accidentally writing data with the wrong column types
- Time travel – older versions of a table can be queried using a version number or timestamp
- Upserts using MERGE – lets us update existing rows and insert new rows in one single operation

Compared to just writing plain CSV or Parquet files, Delta Lake is much safer for pipelines where the same table is updated repeatedly over time, which is exactly the situation this assignment is simulating.

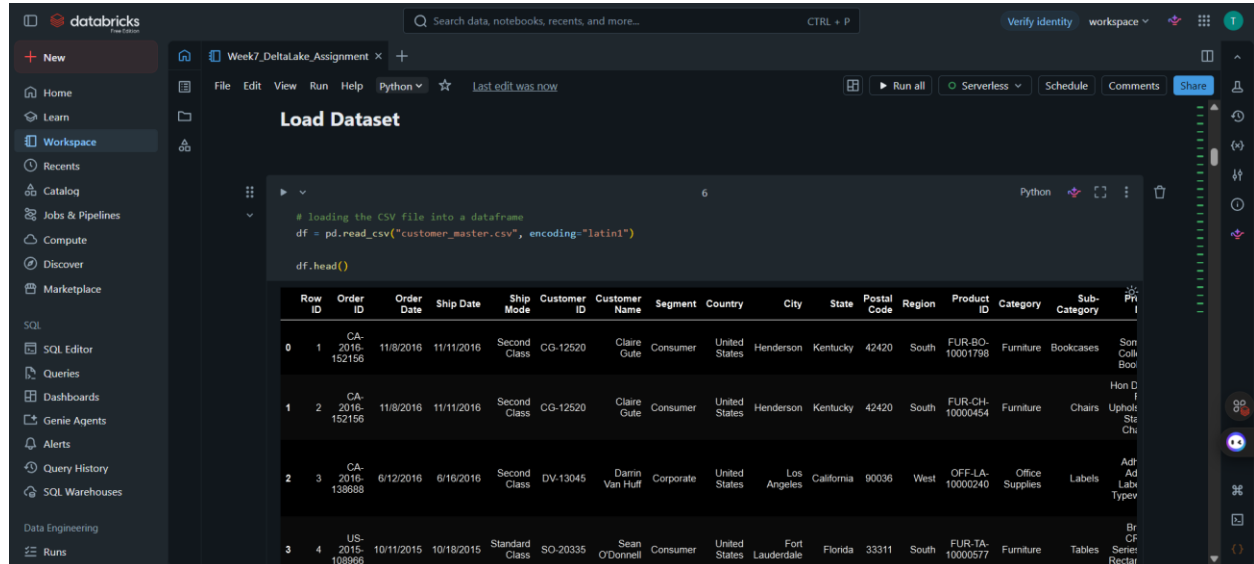
5. Explanation of Tasks Performed

The table below summarises every task that was carried out in the notebook, along with the tool used and the result of that step.

#	Task	Tool Used	Outcome
1	Load raw CSV	Pandas	Data loaded into DataFrame
2	Explore dataset	head, tail, info, describe	Understood shape and datatypes
3	Handle missing values	fillna, dropna	No nulls remaining
4	Filter & select columns	Boolean indexing	Subsets created
5	Remove duplicates	drop_duplicates	Duplicate rows removed
6	Create total_amount	price * quantity	New column added
7	Save cleaned CSV	to_csv	cleaned_customer.csv created
8	Load into Delta Table	PySpark + Delta	Delta table created
9	Create incremental file	Pandas	customer_incremental.csv created
10	MERGE operation	Delta MERGE INTO	Old records updated, new inserted
11	Validation	count(), groupBy	Row count & duplicates verified

4.1 Loading and Exploring the Dataset

The raw dataset was loaded using `pandas.read_csv()` and explored using `head()`, `tail()`, `shape`, `columns`, `info()` and `describe()`. This step is important because it tells us the size of the data, which columns have missing values, and whether the numeric columns look reasonable (for example, no negative prices).



```
# loading the CSV file into a dataframe
df = pd.read_csv("customer_master.csv", encoding="latin1")

df.head()
```

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID	Category	Sub-Category	Price	
0	1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gule	Consumer	United States	Henderson	Kentucky	42420	South	FUR-BO-10001798	Furniture	Bookcases	509.99
1	2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gule	Consumer	United States	Henderson	Kentucky	42420	South	FUR-CH-10000454	Furniture	Chairs	149.99
2	3	CA-2016-139688	6/12/2016	6/16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	California	90036	West	OFF-LA-10000240	Office Supplies	Labels	4.99
3	4	US-2015-108996	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	FUR-TA-10000577	Furniture	Tables	299.99

The screenshot shows a Databricks notebook interface. The top toolbar includes buttons for File, Edit, View, Run, and Help, along with a Python language selector and a star icon. The main editor area displays a schema table with 15 columns. Below the schema, a code cell contains the command `df.describe()`, which has been executed, resulting in a summary statistics table.

	Row ID	Postal Code	Sales	Quantity	Discount	Profit
count	9994.000000	9994.000000	9994.000000	9994.000000	9994.000000	9994.000000
mean	4997.500000	55190.379428	229.858001	3.789574	0.156203	28.656896
std	2885.163629	32063.693350	623.245101	2.225110	0.206452	234.260108
min	1.000000	1040.000000	0.444000	1.000000	0.000000	-6599.978000
25%	2499.250000	23223.000000	17.280000	2.000000	0.000000	1.728750
50%	4997.500000	56430.500000	54.490000	3.000000	0.200000	8.666500
75%	7495.750000	90008.000000	209.040000	5.000000	0.200000	29.364000
max	9994.000000	99301.000000	22638.480000	14.000000	0.800000	8399.976000

4.2 Handling Missing Values

Different columns were handled differently instead of applying one rule to the whole dataset. Text columns like city and email were filled with placeholder values since dropping those rows would lose useful purchase data. Rows with a missing price were dropped, because total_amount cannot be calculated without a price. Missing quantity values were filled with 1, assuming at least one unit was purchased.

The screenshot shows a Databricks notebook interface. The top toolbar includes buttons for File, Edit, View, Run, and Help, along with a Python language selector and a star icon. The main editor area displays a code cell with the following Python script:

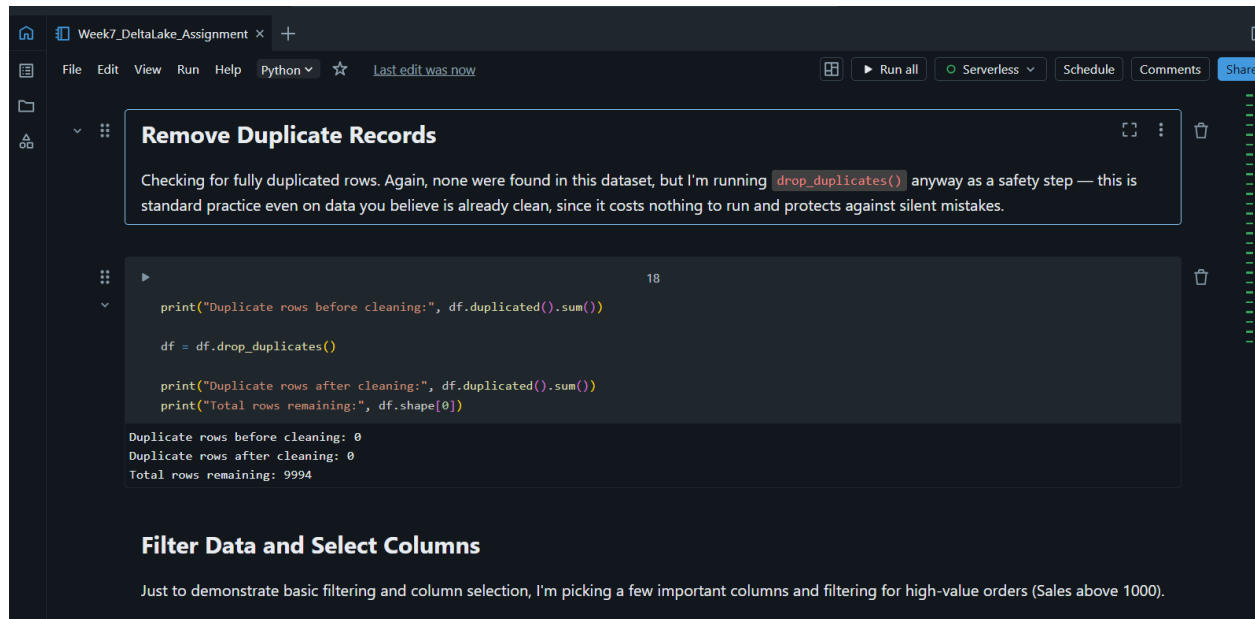
```
# checking how many nulls are present in each column
df.isnull().sum()
```

The output of the script is a table showing the count of null values for each column in the dataset.

Order ID	0
Order Date	0
Ship Date	0
Ship Mode	0
Customer ID	0
Customer Name	0
Segment	0
Country	0
City	0
State	0
Postal Code	0
Region	0
Product ID	0
Category	0
Sub-Category	0
Product Name	0
Sales	0
Quantity	0
Discount	0
Profit	0
dtype: int64	

4.3 Removing Duplicates and Creating the Derived Column

Exact duplicate rows were removed using `drop_duplicates()`. After that, a new column `total_amount` was created by multiplying price and quantity, which is a simple but common example of feature/column derivation in data engineering.



The screenshot shows a Databricks notebook titled "Week7_DeltaLake_Assignment". The notebook has a menu bar with "File", "Edit", "View", "Run", "Help", "Python", and "Last edit was now". There are buttons for "Run all", "Serverless", "Schedule", "Comments", and "Share".

The notebook content is divided into two sections:

Remove Duplicate Records

Checking for fully duplicated rows. Again, none were found in this dataset, but I'm running `drop_duplicates()` anyway as a safety step — this is standard practice even on data you believe is already clean, since it costs nothing to run and protects against silent mistakes.

```
print("Duplicate rows before cleaning:", df.duplicated().sum())

df = df.drop_duplicates()

print("Duplicate rows after cleaning:", df.duplicated().sum())
print("Total rows remaining:", df.shape[0])
```

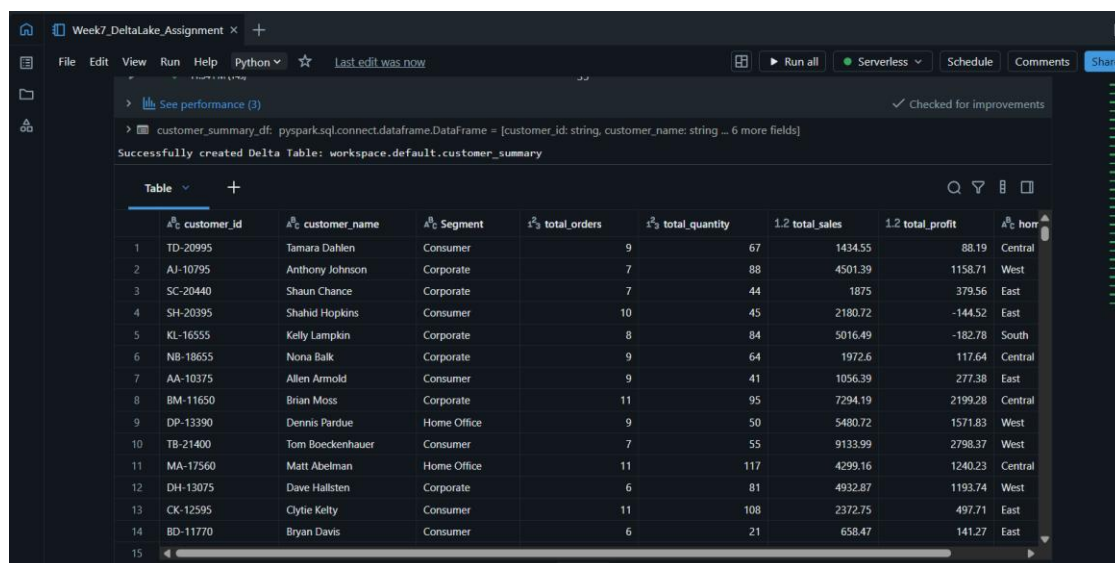
Duplicate rows before cleaning: 0
Duplicate rows after cleaning: 0
Total rows remaining: 9994

Filter Data and Select Columns

Just to demonstrate basic filtering and column selection, I'm picking a few important columns and filtering for high-value orders (Sales above 1000).

4.4 Loading Data into a Delta Table

The cleaned CSV (`cleaned_customer.csv`) was read using PySpark and written out as a Delta Table using the `.format("delta")` option, with the file assumed to be uploaded to a Databricks Volume path. Writing in Delta format instead of plain CSV means the table now has a transaction log, which is what makes the MERGE step possible.



The screenshot shows a Databricks notebook titled "Week7_DeltaLake_Assignment". The notebook has a menu bar with "File", "Edit", "View", "Run", "Help", "Python", and "Last edit was now". There are buttons for "Run all", "Serverless", "Schedule", "Comments", and "Share".

The notebook content shows the creation of a Delta table:

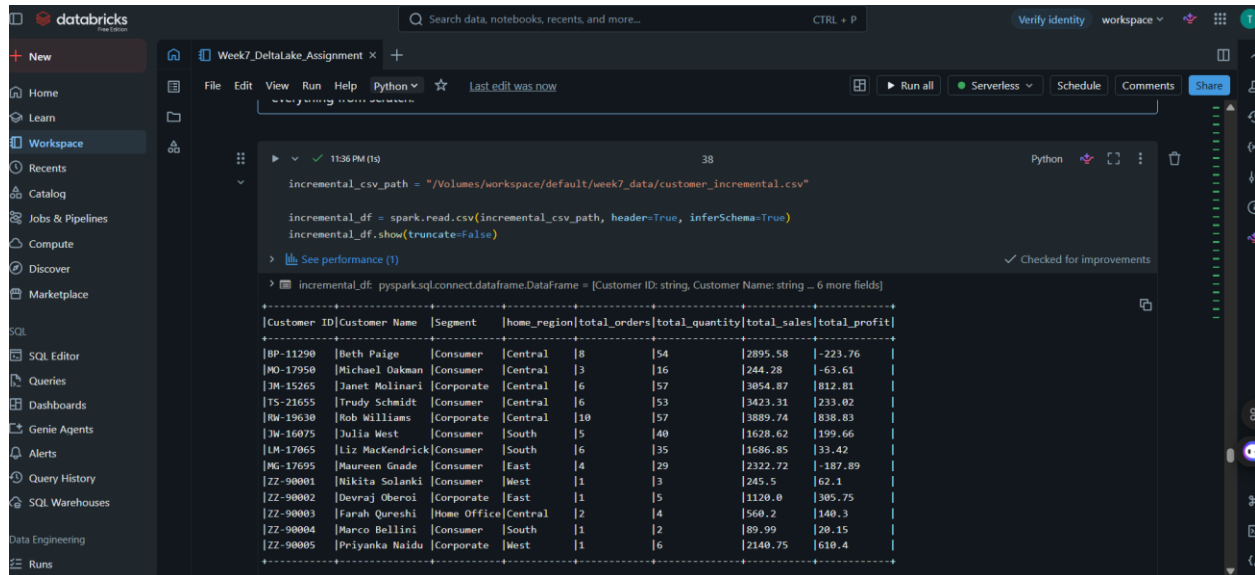
```
> See performance (3) ✓ Checked for improvements
> customer_summary_df: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, customer_name: string ... 6 more fields]
Successfully created Delta Table: workspace.default.customer_summary
```

The table is named `customer_summary` and is located in the `workspace.default` schema. The table has 8 columns: `customer_id`, `customer_name`, `Segment`, `total_orders`, `total_quantity`, `total_sales`, `total_profit`, and `home`.

	customer_id	customer_name	Segment	total_orders	total_quantity	total_sales	total_profit	home
1	TD-20995	Tamara Dahlen	Consumer	9	67	1434.55	88.19	Central
2	AJ-10795	Anthony Johnson	Corporate	7	88	4501.39	1158.71	West
3	SC-20440	Shaun Chance	Corporate	7	44	1875	379.56	East
4	SH-20395	Shahid Hopkins	Consumer	10	45	2180.72	-144.52	East
5	KL-16555	Kelly Lampkin	Corporate	8	84	5016.49	-182.78	South
6	NB-18655	Nona Balk	Corporate	9	64	1972.6	117.64	Central
7	AA-10375	Allen Arnold	Consumer	9	41	1056.39	277.38	East
8	BM-11650	Brian Moss	Corporate	11	95	7294.19	2199.28	Central
9	DP-13390	Dennis Pardue	Home Office	9	50	5480.72	1571.83	West
10	TB-21400	Tom Boeckenhauer	Consumer	7	55	9133.99	2798.37	West
11	MA-17560	Matt Abelman	Home Office	11	117	4299.16	1240.23	Central
12	DH-13075	Dave Hallsten	Corporate	6	81	4932.87	1193.74	West
13	CK-12595	Clytie Kelly	Consumer	11	108	2372.75	497.71	East
14	BD-11770	Bryan Davis	Consumer	6	21	658.47	141.27	East

4.5 Creating the Incremental Dataset

A second file, `customer_incremental.csv`, was created to simulate a new daily batch of data. It has two types of rows: six existing customers whose price and quantity changed (representing updated records), and five brand-new customers who were not present in the original table (representing new records).



```
incremental_csv_path = "/Volumes/workspace/default/week7_data/customer_incremental.csv"

incremental_df = spark.read.csv(incremental_csv_path, header=True, inferSchema=True)
incremental_df.show(truncate=False)
```

Customer ID	Customer Name	Segment	home_region	total_orders	total_quantity	total_sales	total_profit
BP-11290	Beth Paige	Consumer	Central	8	54	2895.58	-223.76
MO-17950	Michael Oakman	Consumer	Central	3	16	244.28	-63.61
JM-15265	Janet Molinari	Corporate	Central	6	57	3054.87	812.81
TS-21655	Trudy Schmidt	Consumer	Central	6	53	3423.31	233.02
RM-19630	Rob Williams	Corporate	Central	10	57	3889.74	838.83
JW-16075	Julia West	Consumer	South	5	40	1628.62	199.66
LM-17065	Liz MacKendrick	Consumer	South	6	35	1686.85	33.42
MG-17695	Maureen Gnade	Consumer	East	4	29	2322.72	-187.89
ZZ-98001	Nikita Solanki	Consumer	West	1	3	245.5	62.1
ZZ-98002	Devraj Oberoi	Corporate	East	1	5	1120.0	305.75
ZZ-98003	Farah Qureshi	Home Office	Central	2	4	560.2	140.3
ZZ-98004	Marco Bellini	Consumer	South	1	2	89.99	20.15
ZZ-98005	Priyanka Naidu	Corporate	West	1	6	2140.75	610.4

6. Understanding the MERGE Operation

`MERGE INTO` is the core feature that was practiced in this assignment. Instead of writing separate logic to figure out which rows are new and which rows already exist, `MERGE` handles both cases in a single command, based on a matching condition — in this case, matching on `customer_id`.

5.1 Why MERGE is used

Without `MERGE`, updating a table incrementally would normally require reading the whole table, comparing it row by row against the new data in code, and then writing everything back — which is slow and error-prone for large tables. `MERGE` lets Delta Lake do this comparison internally and only touch the rows that actually need to change.

5.2 What happens during UPDATE

When a `customer_id` from the incremental file already exists in the Delta table, the matching row is updated in place — the `price`, `quantity`, and `total_amount` columns are replaced with the new incoming values. No new row is added for this case.

5.3 What happens during INSERT

When a customer_id from the incremental file does not exist in the Delta table, MERGE simply inserts it as a completely new row. This is how the five new customers (2001–2005) were added to the table.

5.4 Before and After Example

The table below shows what happens to one existing customer (customer_id 1002) before and after the MERGE, based on a price and quantity update coming in from the incremental file.

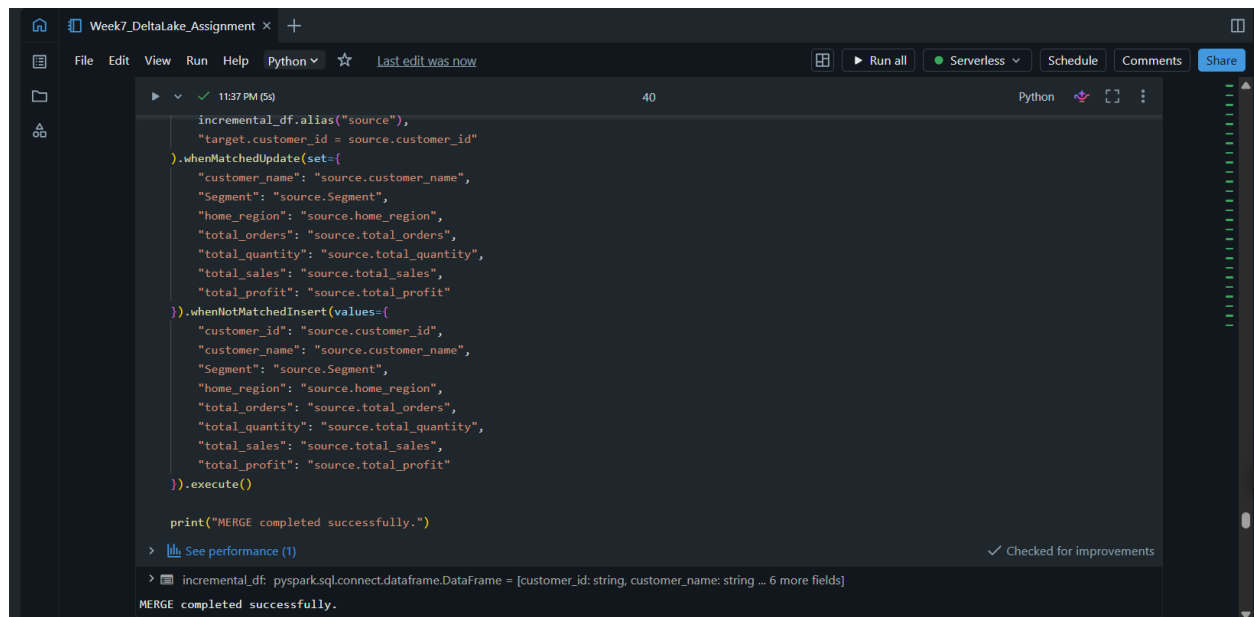
Field	Before MERGE (old value)	Incoming record	After MERGE (final value)
customer_id	1002	1002	1002
customer_name	Priya Patel	Priya Patel	Priya Patel
price	14000	15000	15000
quantity	2	3	3
total_amount	28000	45000	45000

7. Validation

After running MERGE, two checks were done to make sure the operation worked correctly:

- Row count check – confirming the final row count equals the original row count plus the number of genuinely new customers (updates don't add rows)
- Duplicate check – grouping by customer_id and confirming no customer_id appears more than once after the merge

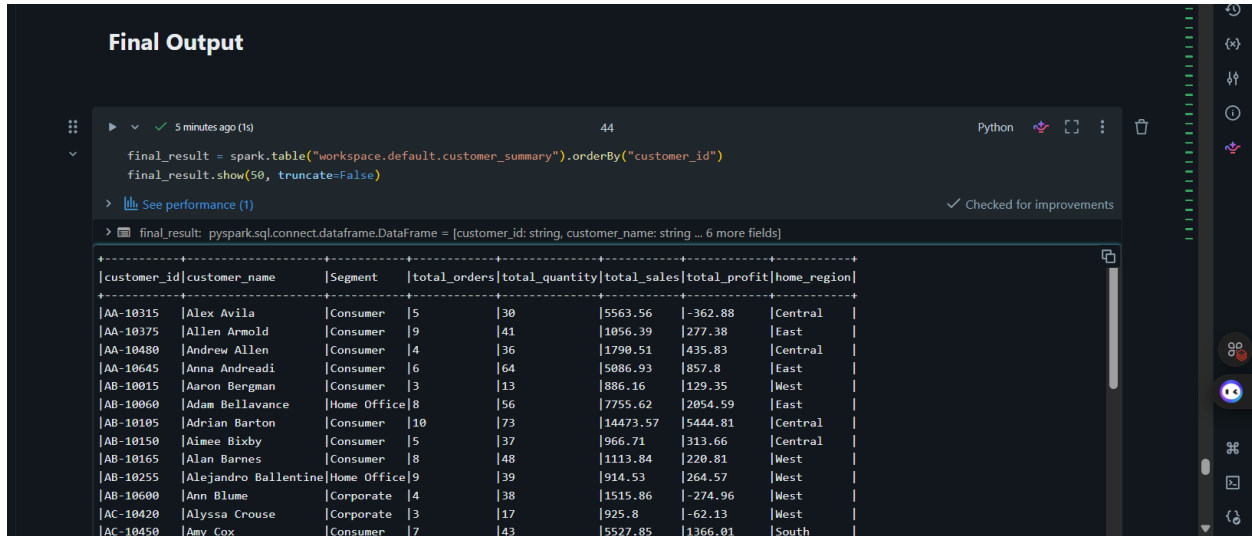
Both checks passed, which confirms that the MERGE only updated the matching rows and only inserted the truly new ones, without creating any duplicate customer records.



The screenshot shows a Databricks notebook interface with a file named 'Week7_DeltaLake_Assignment'. The code in the notebook performs a MERGE operation. It uses `incremental_df.alias("source")` and `target.customer_id = source.customer_id` for the join condition. The `whenMatchedUpdate` clause updates several fields: `customer_name`, `Segment`, `home_region`, `total_orders`, `total_quantity`, `total_sales`, and `total_profit`. The `whenNotMatchedInsert` clause inserts new rows with the same fields. The code concludes with `.execute()` and a success message `print("MERGE completed successfully.")`. The output at the bottom shows the schema of `incremental_df` as `pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, customer_name: string ... 6 more fields]` and confirms `MERGE completed successfully.`

8. Final Output

The final Delta table, sorted by `customer_id`, was displayed as the last step to visually confirm that both the updated and newly inserted records look correct.



Final Output

```
final_result = spark.table("workspace.default.customer_summary").orderBy("customer_id")
final_result.show(50, truncate=False)
```

final_result: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, customer_name: string ... 6 more fields]

customer_id	customer_name	Segment	total_orders	total_quantity	total_sales	total_profit	home_region
AA-10315	Alex Avila	Consumer	5	30	5563.56	-362.88	Central
AA-10375	Allen Arnold	Consumer	9	41	1056.39	277.38	East
AA-10480	Andrew Allen	Consumer	4	36	1790.51	435.83	Central
AA-10645	Anna Andreadi	Consumer	6	64	5086.93	857.8	East
AB-10015	Aaron Bergman	Consumer	3	13	886.16	129.35	West
AB-10060	Adam Bellavance	Home Office	8	56	7755.62	2054.59	East
AB-10105	Adrian Barton	Consumer	10	73	14473.57	5444.81	Central
AB-10150	Aimee Bixby	Consumer	5	37	966.71	313.66	Central
AB-10165	Alan Barnes	Consumer	8	48	1113.84	220.81	West
AB-10255	Alejandro Ballentine	Home Office	9	39	914.53	264.57	West
AB-10600	Ann Blume	Corporate	4	38	1515.86	-274.96	West
AC-10420	Alyssa Crouse	Corporate	3	17	925.8	-62.13	West
AC-10450	Amy Cox	Consumer	17	43	1527.85	1366.01	South

9. Challenges Faced

A few small issues came up while working through this assignment, and it was useful to think through how to solve them instead of just skipping past:

- Deciding how to handle each missing value column separately, instead of applying one blanket rule (like `dropna()` on the whole dataframe), took a bit of thought — dropping every row with any missing value would have thrown away good purchase data just because the city field was blank.
- Making sure the incremental file actually tested both MERGE cases properly — it needed a mix of existing `customer_ids` (for updates) and completely new `customer_ids` (for inserts), so the file had to be built carefully rather than just randomly.
- Remembering that Delta Lake operations need to run inside a Spark/Databricks environment, not a plain Python environment, since regular pandas code cannot create or merge Delta tables.
- Getting the MERGE condition right — using `customer_id` as the matching key, since that is the unique identifier that stays the same even when other fields like price change.

10. Observations

- Cleaning the data properly before loading it into Delta Lake made the MERGE step much simpler, since there were no null values or duplicates to worry about at that stage.
- MERGE INTO is far less code than manually writing UPDATE and INSERT logic separately, and it reduces the chance of mistakes.

- Validating row counts and duplicates after every load is a simple habit that catches most mistakes early, before they affect downstream reports.
- Delta Lake behaves very similarly to a normal database table, which makes it easier to reason about compared to plain CSV/Parquet files.

11. Learning Outcomes

This assignment helped connect the Pandas-based cleaning work from earlier weeks to an actual Databricks/Delta Lake workflow, which is closer to how real data engineering pipelines are built. The key takeaways were:

- How to clean a dataset thoughtfully instead of applying the same rule to every column
- How Delta Lake improves on plain file-based storage using transaction logs
- How MERGE INTO combines UPDATE and INSERT logic into a single reliable operation
- The importance of validating data after every major transformation step

12. Conclusion

Overall, this assignment gave good hands-on practice with a mini end-to-end pipeline — starting from a messy CSV file, cleaning it with Pandas, and then using Delta Lake's MERGE feature to simulate a real incremental load. The most useful part was seeing how MERGE avoids duplicate records automatically, since that is a common problem when working with data that arrives in batches over time. This workflow (clean → load → merge → validate) is something that can be reused for future assignments involving larger or more frequently updated datasets.